

AI-901

Exam Terminology & Concepts Reference

Section-wise glossary companion for the mock tests and the real exam

Study this before attempting any timed test.

Definitions, contrasts, and classic exam traps — no test questions or answers inside.

Faculty Study Kit · original content from public Microsoft Learn documentation

A section-wise glossary of every technical term and concept you need for the AI-901 mock test and the real exam. Study this before attempting the tests — it defines the vocabulary and the classic confusions, without giving away any test answers.

Section 1 — The Microsoft Foundry Platform

Microsoft Foundry — Azure's unified platform for building, deploying, evaluating and managing AI applications and agents. It is the successor branding of *Azure AI Studio* and the center of the AI-901 exam. Everything below — catalogs, deployments, playgrounds, agents, evaluations — lives here.

Foundry portal — the no-code web experience at ai.azure.com. Used to browse the model catalog, deploy models, test in playgrounds, build agents, and run evaluations. Remember the address; distractors will offer Office, Azure ML Studio, or invented URLs.

Hub — the *top-level workspace*. It holds shared security settings, connections, and compute that all projects underneath it inherit. Think: the building.

Project — a workspace *inside a hub* for building one specific solution. It contains your deployments, agents, files, and evaluations. Think: an office inside the building.

Deployment — a *callable instance of a model* inside a project. A model sitting in the catalog cannot be called; only after **you deploy it** (and name the deployment) does it get an endpoint your code can reach.

Model catalog — the searchable library of models from Microsoft, OpenAI, Meta, Mistral, Cohere, and community providers. You **discover, filter, compare, and deploy** from here. Microsoft-sold models bill through your Azure subscription.

Connections — configured links from a hub/project to external resources (data sources, services) used for grounding and integration. Not where you deploy models.

Playground — the interactive test surface for a deployed model or agent. It is for *experimentation*: trying prompts, comparing models, tuning parameters — then carrying the working configuration into code. It does not replace deployment or the API.

Hierarchy to memorize: Hub (shared security/resources) → Project (your solution) → Deployment (a callable model). Separately, in Azure itself: Tenant (security/Entra ID) → Subscription (billing) → Resource group (organization) → Resources.

Section 2 — Calling Models from Code

Endpoint — the HTTPS URL of your deployed model/service. It answers "*where* do I call?"

Credential — proof you may call the endpoint. Two forms:

Credential	What it is	When used
API key	A shared secret string	Quick starts, demos
Microsoft Entra ID token (e.g. via <code>DefaultAzureCredential</code>)	Identity-based, <i>keyless</i> auth resolved from the running environment (developer sign-in, managed identity)	Recommended for production — no secret stored in code

Deployment name — the third thing every call needs. In Azure SDK calls the parameter is called `model` , but for Azure deployments you pass **the name you gave your deployment**, *not* the base model's catalog name. Passing the catalog name when your deployment is named differently is the classic cause of a `DeploymentNotFound` (404) error.

The three-item rule — every model call needs: **endpoint + credential + deployment name**.

Statelessness — chat models have **no memory between calls**. What feels like conversation memory is your *application* re-sending the previous user and assistant messages with each new request. Consequence: longer conversations consume **more** tokens, not fewer.

Response shape — a chat completion returns a list of `choices` ; each choice holds a `message` with a `role` and `content` . The assistant's reply text is at `response.choices[0].message.content` .

`finish_reason` — why generation stopped:

Value	Meaning	Typical fix
<code>stop</code>	Natural completion	—
<code>length</code>	Output hit the max_tokens cap → answers cut off mid-sentence	Raise <code>max_tokens</code>
<code>content_filter</code>	Blocked by the safety system	Review filter settings / prompt

Section 3 — Tokens, Parameters & Prompt Engineering

Token — the unit of text a model reads and writes (~a word-piece; roughly 4 English characters).

Billing, rate limits, context windows, and `max_tokens` are all measured in tokens. A token is

not a security key (that's a credential) and *not* a meaning vector (that's an embedding).

Context window — the maximum number of tokens (prompt + history + response) the model can consider in one call. Nothing you configure at request time enlarges it — designs like RAG exist precisely to work *within* it.

Request parameters — the most-tested table on the exam

Parameter	Controls	Common trap
<code>temperature</code>	Sampling randomness — lower = focused/deterministic, higher = creative/varied	It does not control length
<code>top_p</code>	Sampling breadth (nucleus sampling) — an alternative randomness control	Same family as temperature
<code>max_tokens</code>	Maximum output length (and therefore cost)	It does not control creativity
<code>frequency_penalty</code>	Discourages tokens that already appeared often → reduces repetition within a response	
<code>presence_penalty</code>	Encourages the model to move to new topics	Related to, but different from, frequency penalty
<code>seed</code>	Makes sampling reproducible across runs when inputs and settings are identical — for regression testing (often with temperature 0)	
<code>stop</code>	Strings at which generation halts	Not a safety feature

The #1 confusion pair: *temperature vs max_tokens*. Randomness vs length. If a scenario mentions "inconsistent answers" think temperature; if it mentions "answers too long / cut off / expensive" think max_tokens.

Message roles

Role	Carries
system	Persona, rules, scope limits, output format requirements , and grounding data — set <i>before</i> the conversation and steering every turn
user	The actual request/question
assistant	The model's own previous replies (resent for context)
tool / function	Results of tool executions returned to the model

Few-shot prompting — including one or more worked input→output examples in the prompt so the model imitates the pattern. The most reliable way (together with system-message format rules) to force strict output structures such as JSON with specific keys.

Section 4 — Model Types & Choosing Between Them

Model type	What it does	Choose it when
LLM (large language model)	General text understanding & generation	Broad, capable text tasks
SLM (small language model, e.g. Phi family, "mini" models)	Compact, fast, cheap	Simple, high-volume tasks (e.g. classifying millions of short texts) and constrained devices
Reasoning models (o-series)	Extended multi-step logical reasoning	Complex math / multi-step logic
Multimodal model	Understands more than one input type — e.g. an image + a question; some support audio in / audio out in real time	Ad-hoc questions about an image; single-model voice assistants
Image-generation model (DALL-E-3 / gpt-image family)	Creates new images from a text prompt	"Generate a picture of..."
Video-generation model (sora family)	Creates video from prompts — runs as an asynchronous job you poll	Video creation
Embeddings model (text-embedding family)	Converts text into meaning vectors — it never generates prose	Semantic search, RAG retrieval

Trap pair: a *multimodal* model **understands** images you give it; an *image-generation* model **creates** images. Different tools for different directions. **Trap pair 2:** big/reasoning models for hard problems; **small models for cheap, fast, simple, high-volume** work.

Section 5 — Deployment Types & Hosting Options

Option	Pricing / hosting	Best for
Standard	Pay-per-token, regional	General interactive use
Global Standard	Pay-per-token, routed across Microsoft's global capacity	Spiky/unpredictable traffic needing best availability without reserved capacity
Provisioned Throughput (PTU)	Reserved, dedicated capacity	Steady high-volume production needing predictable latency
Batch	Asynchronous bulk processing at reduced cost	Huge offline jobs where nobody waits (e.g. millions of documents overnight)
Serverless API / Models-as-a-Service (MaaS)	Call partner/open models (Llama, Mistral...) via API, no infrastructure to manage	Using open models without hosting anything
Managed compute	Model hosted on dedicated VMs you control in Azure	Custom/fine-tuned models and strict control/compliance requirements

Also remember the general deployment considerations: **deployment type** (where data is processed / how you pay), **version** (+ auto-update policy), **rate limits** (tokens-per-minute throughput; exceeding causes throttling), **guardrails** (responsible-AI policies).

Section 6 — Embeddings, Vector Search & RAG

Embedding — a numeric **vector representing the meaning** of text (or other content). Similar meanings → nearby vectors. Powers semantic search, recommendations, and RAG retrieval.

Vector index — a store (e.g. **Azure AI Search**) that retrieves items by vector similarity instead of exact keywords → **semantic search** ("find by meaning").

Chunking — splitting long documents into smaller passages before embedding, so retrieval returns focused, context-window-sized pieces.

Hallucination — output that *sounds plausible but is factually wrong or fabricated*. It happens because LLMs predict likely next tokens from learned patterns — they do not look facts up. It is **not** a network error, a filter refusal, or a truncated answer.

Grounding — supplying authoritative source material in the prompt so the model answers from facts rather than memory.

RAG (Retrieval-Augmented Generation) — the standard grounding architecture:

1. Chunk and **embed** your documents into a **vector index** (done ahead of time).
2. At query time, embed the user's question and **retrieve** the most similar chunks.
3. Inject those chunks into the prompt (grounding) and let the model answer **from them**.

RAG is the standard mitigation for hallucinations *and* the standard way to answer over content far larger than the context window. Raising temperature makes hallucination worse; raising max_tokens only changes length — neither is a fix.

Section 7 — Agents & the Foundry Agent Service

The agent equation: **agent** = **model** + **instructions** + **tools** (+ optional knowledge).

- **Model** — language understanding and reasoning.
- **Instructions** — the agent's system-prompt-like job description: persona, task, rules.
- **Tools** — capabilities the agent can invoke:

Tool	What it enables
File search / knowledge	Retrieves passages from uploaded documents to ground answers — the agent-world equivalent of RAG
Code interpreter	Writes and runs Python in a sandbox — load a CSV, compute statistics, draw charts
Function calling	Lets the agent invoke your registered APIs/functions to take real actions (book, create, update)

How function calling actually works — the model never executes your code. It emits a *structured function call* (function name + JSON arguments); your application or the Agent Service executes the function **outside the model** and returns the result so the model can finish its answer.

Agent Service lifecycle vocabulary:

Object	Meaning
Agent	The definition (model + instructions + tools)
Thread	A conversation container holding messages
Message	One user or assistant turn added to a thread
Run	One <i>execution</i> of the agent over a thread — the agent reasons, possibly calls tools, and writes its reply back to the thread

The natural order of operations follows from the definitions: you must have an agent and a thread before you can add messages; a run executes over what the thread contains; replies are read from the thread after the run.

Section 8 — Safety, Guardrails & Responsible AI

Content filters (Azure AI Content Safety) — screen prompts and completions for harmful **content categories** (hate, violence, sexual, self-harm) with **adjustable severity thresholds** per category. Legitimate specialist domains (e.g. medical education) may need the thresholds reviewed and tuned — that is what the configuration exists for.

Prompt shields — a *different* protection: they defend against **prompt-injection and jailbreak attacks** — attempts to hijack the model's instructions. Filters police *content*; shields police *attacks*. They are not the same feature.

Prompt injection / jailbreak — adversarial input crafted to override the system's rules.

The six Responsible AI principles (with the classic traps)

Principle	One-liner	Trap to avoid
Fairness	No biased <i>outcomes</i> across groups — identical profiles must get consistent decisions	Don't confuse with inclusiveness
Reliability & safety	The system behaves consistently and safely, even in unexpected conditions; test rigorously	Don't confuse with accountability
Privacy & security	Protect data in training <i>and</i> operation	
Inclusiveness	AI should empower everyone, including people of all abilities (accessibility)	It's about who benefits, not about decision bias
Transparency	Purpose, behavior and limitations must be understandable	
Accountability	People/organizations stay answerable — governance boards, named owners, escalation and override processes	It's about people, not system robustness

Section 9 — Evaluation & Quality

Groundedness — measures whether a response is actually **supported by the supplied source data**. The essential pre-release metric for a RAG application: a fluent answer can still be fabricated.

Fluency — language quality: grammar, readability. Says nothing about truth.

Coherence — logical flow and consistency of the response. Also says nothing about truth.

Relevance — how well the answer addresses the question asked.

Token usage — a **cost/consumption** measure, not a quality metric.

Reproducibility — for regression testing, fix the **seed** and keep inputs/parameters identical (often with temperature 0) so nightly runs are comparable.

Section 10 — Speech

Capability	Direction	Example uses
Speech recognition (speech-to-text, STT)	Audio → text	Transcription of recordings/meetings, captions/subtitles, voice commands
Speech synthesis (text-to-speech, TTS)	Text → audio	Reading answers aloud, IVR, screen readers, alerts
Speech translation	Spoken language A → text/speech in language B	Multilingual kiosks and assistants
Speaker recognition	Identifies/verifies who is speaking	Voice verification — note it does <i>not</i> transcribe what was said

Neural voices — natural-sounding synthetic voices used by TTS.

SSML (Speech Synthesis Markup Language) — markup for fine-tuning a synthetic voice's pronunciation, pitch, rate and prosody. It tunes *how* speech sounds; it doesn't translate or transcribe anything.

Pipeline vocabulary — recognition: audio capture → feature extraction → **acoustic model** (features → **phonemes**, the smallest units of sound) → **language model** (most probable word sequence) → post-processing. Synthesis: text → **normalization** (expand "Dr.", "3:00pm") → phonemes → **prosody** (natural cadence) → waveform.

Realtime / audio multimodal models — a single model that accepts speech in and produces speech out (e.g. GPT-4o-audio-style, or Foundry's Voice Live experience), replacing the chained STT → text model → TTS pipeline, with continuous conversation and interruption handling.

Section 11 — Information Extraction & Azure Content Understanding

Information extraction — analyzing unstructured content (documents, images, audio, video) to pull out **structured fields and values**.

OCR (optical character recognition) — converts *images of text* into machine-readable text. Reading only — no understanding of what the values mean.

Azure Content Understanding — Foundry's single **multimodal extraction service**. It goes beyond OCR by understanding document structure and **mapping extracted data to a defined schema**, returning structured **JSON with confidence scores**. It covers the scenarios that older separate products (Document Intelligence / Form Recognizer for documents, Video Indexer for media) handled.

Analyzer — the artifact that defines *how content is processed and what structured data comes back*. **Predefined analyzers** cover common documents (invoices, receipts, contracts) and call/video scenarios; **custom analyzers** are built by defining your own **field schema** from sample content.

Field schema — the explicit list of fields (e.g. `vendor_name` , `invoice_date` , `total_amount`) an analyzer extracts. This — not a system message or a filter — is what determines the returned fields.

Confidence score — per-field certainty value enabling human-in-the-loop review: auto-accept high confidence, route low confidence to a person.

Asynchronous analysis — with the SDK you submit content and then **poll a URL until the job completes**; results are not returned in the same request.

The repeatability rule of thumb: the *same structured fields, extracted repeatedly, at scale* → build a Content Understanding **analyzer**. A *one-off question about a single item* (one photo, one page) → just **prompt a multimodal model**. Ad-hoc prompting doesn't scale to pipelines; building an analyzer for one photo is overkill.

Section 12 — Legacy → Current Service Map

Legacy product (AI-900 era)	Current capability (AI-901 era)
Azure AI Studio	Microsoft Foundry
LUIS (Language Understanding)	Conversational Language Understanding (CLU)
QnA Maker	Custom question answering (QnA Maker itself is retired)
Form Recognizer / Document Intelligence scenarios	Azure Content Understanding
Video Indexer scenarios	Azure Content Understanding

Section 13 — Commonly Confused Pairs (last-minute revision)

Pair	The distinction in one line
temperature vs max_tokens	randomness vs output length
frequency vs presence penalty	stop repeating vs move to new topics
token vs embedding vs seed	text unit vs meaning vector vs reproducibility knob
content filter vs prompt shield	harmful content categories vs injection/jailbreak attacks
hub vs project vs deployment	shared workspace vs solution workspace vs callable model
base model name vs deployment name	catalog label vs the name <i>your code must call</i>
multimodal vs image-generation model	understands images vs creates images
STT vs TTS vs translation vs speaker recognition	transcribe vs speak vs change language vs identify the voice
groundedness vs fluency/coherence	supported by sources vs merely well-written
PTU vs Global Standard vs Batch	reserved capacity vs global pay-per-token vs offline bulk
Serverless (MaaS) vs managed compute	no infrastructure vs dedicated VMs you control
fairness vs inclusiveness	unbiased outcomes vs empowering all abilities
reliability & safety vs accountability	the <i>system</i> behaves safely vs <i>people</i> are answerable
hallucination vs filter block vs length cut-off	fabricated content vs safety refusal vs <code>finish_reason: length</code>
analyzer vs one-off multimodal prompt	repeatable schema extraction vs single ad-hoc question

Original reference material authored from public Microsoft Learn documentation for the AI-901 faculty study kit. Definitions only — no test questions or answers are reproduced here.